Universidad Cuenca del Plata - Sede Posadas

Paradigmas y Lenguajes de Programación III

Actividad N°3: (21/10) - Prueba de Carga y Blindaje de Endpoints en Vivo

Alumnos: Noguera David y Viarengo Gonzalo

Proyecto: Savage Grill (API REST de gestión gastronómica)

Fecha de realización: 21/10/2026

1. Descripción

Se realizaron las tres pruebas pedidas en la consigna sobre la API del proyecto Savage Grill, usando Postman contra el servidor local (`http://localhost:8080/api/v1`), con el repositorio posicionado en la rama de Git `integracion/back-front`. Las pruebas comprueban la latencia de la API bajo carga recurrente, el rechazo de entradas maliciosas y la configuración de los encabezados HTTP de seguridad.

Orden en que se ejecutaron: Prueba 1 (carga), Prueba 2 (inyección de script) y Prueba 3 (headers de seguridad). La Prueba 3 se inspeccionó sobre la respuesta del mismo request de la Prueba 2.

2. Resumen de resultados

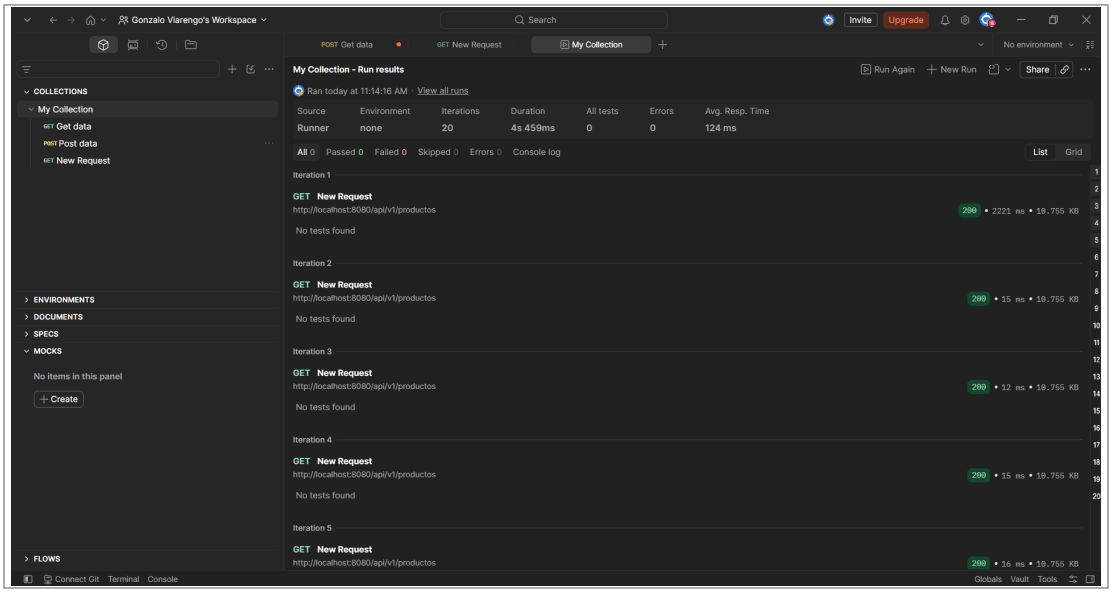
Prueba	Resultado esperado	Resultado obtenido
1. Test de carga concurrente	Disparar 20 peticiones y medir la latencia media y la tasa de éxito.	20 iteraciones con 0 errores (100% de éxito) y una latencia media de 124 ms.
2. Sanitización y Error 400	Inyectar caracteres maliciosos y verificar el rechazo con 400 Bad Request, sin error 500.	400 Bad Request con un mensaje de validación. Las 2 verificaciones pasaron (PASSED): el servidor no devolvió error 500.
3. Headers de seguridad	Encabezados HTTP anti-sniffing y anti-clickjacking activos en el servidor.	Se verificó en la respuesta la presencia de `X-Content-Type-Options: nosniff` y `X-Frame-Options: DENY`.

3. Prueba 1: test de carga concurrente

Se creó en Postman un request `GET /productos` (el catálogo de productos) y se lo ejecutó con la herramienta Runner configurada en 20 iteraciones, para estresar el endpoint y verificar su robustez. El Runner lanza las iteraciones una detrás de otra, sin pausa de por medio, lo que genera una carga recurrente sobre el servidor local.

Esperado: medir la latencia media y la tasa de éxito de las 20 peticiones. Obtenido: 20 iteraciones completadas con 0 errores, una duración total de 4 s 459 ms y una latencia media de 124 ms. Todas las respuestas visibles fueron 200 OK con el mismo tamaño (10,755 KB).

La primera iteración tardó 2221 ms, mientras que las iteraciones siguientes que se ven en la captura respondieron entre 12 y 16 ms. Ese salto se explica por la caché en memoria RAM implementada previamente: la primera consulta arma la respuesta y las siguientes se sirven desde la caché. Por eso la media de 124 ms está influida por esa primera respuesta y no refleja el tiempo habitual de las demás. El request no tiene scripts de test (por eso el Runner muestra 0 tests), así que el criterio de éxito fue el estado 200 y el contador de errores en 0.



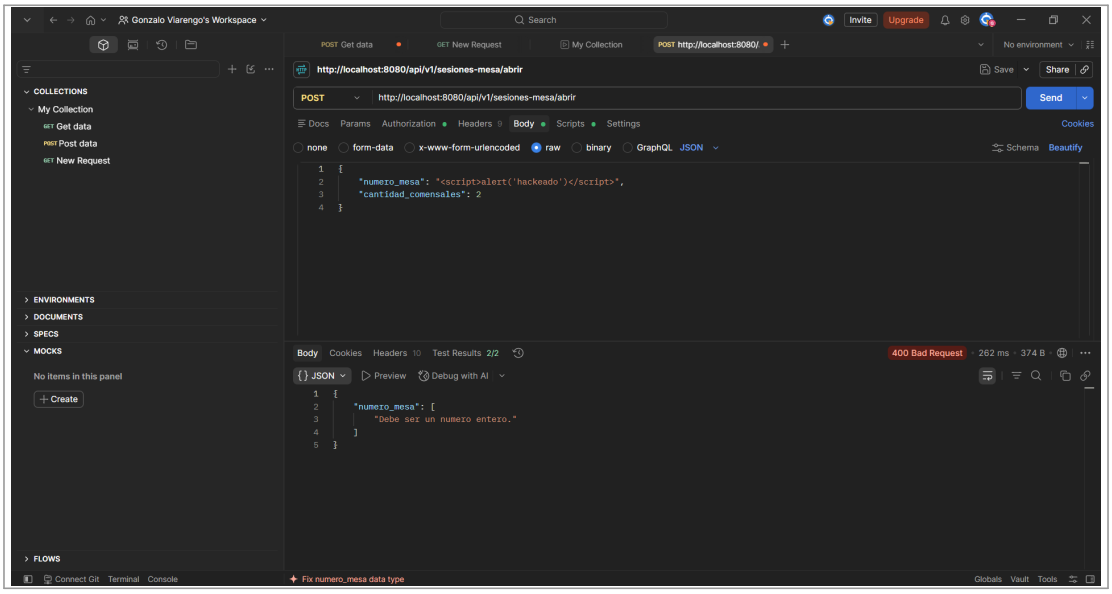
Captura 1. Runner de Postman: 20 iteraciones, 0 errores y latencia media de 124 ms.

4. Prueba 2: sanitización y Error 400

Se envió un `POST` a `/sesiones-mesa/abrir` (apertura de mesas) inyectando código Javascript malicioso en el campo `numero_mesa` del cuerpo del JSON, para intentar vulnerar el sistema. El objetivo era comprobar que la API no procesa la entrada maliciosa y que responde con un error controlado, no con un error del servidor. El cuerpo enviado fue:

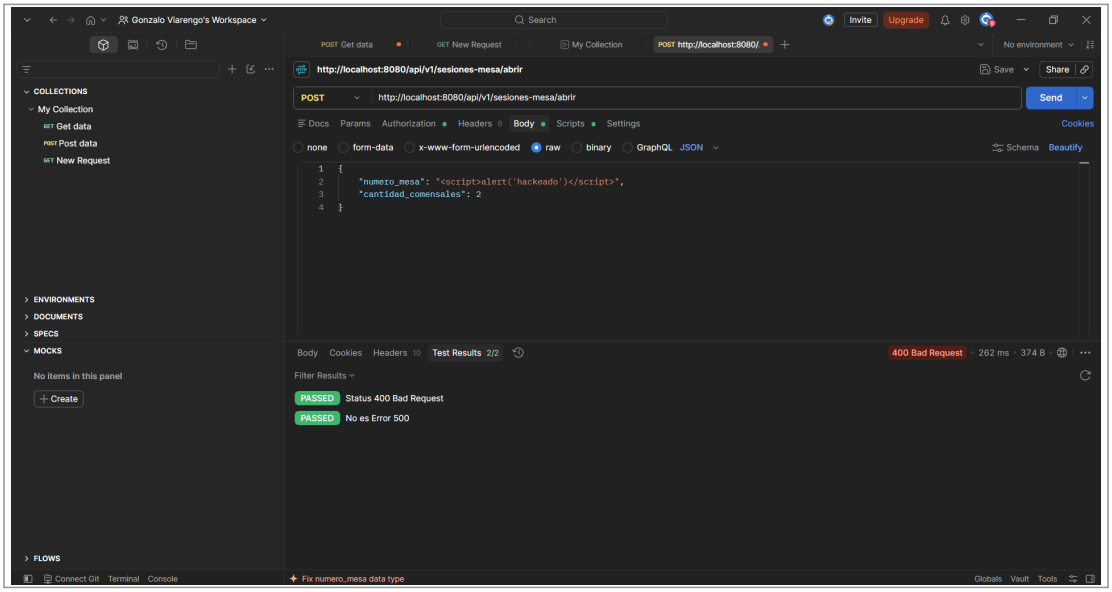
```
{
  "numero_mesa": "<script>alert('hackeado')</script>",
  "cantidad_comensales": 2
}
```

Esperado: status 400 Bad Request con un mensaje explicativo y sin error 500. Obtenido: 400 Bad Request con el mensaje `"Debe ser un numero entero."` para el campo `numero_mesa`.



Captura 2. POST /sesiones-mesa/abrir con un script en numero_mesa: 400 Bad Request y mensaje de validación.

El servidor validó la entrada y rechazó la petición: el campo solo acepta un número entero, de modo que el texto con el script se descarta antes de llegar a la lógica de negocio y a la base de datos. Los scripts de prueba de Postman verificaron dos cosas: que el status sea 400 Bad Request y que no sea un error 500. Las dos quedaron en PASSED, lo que confirma que la API manejó la excepción de forma controlada.

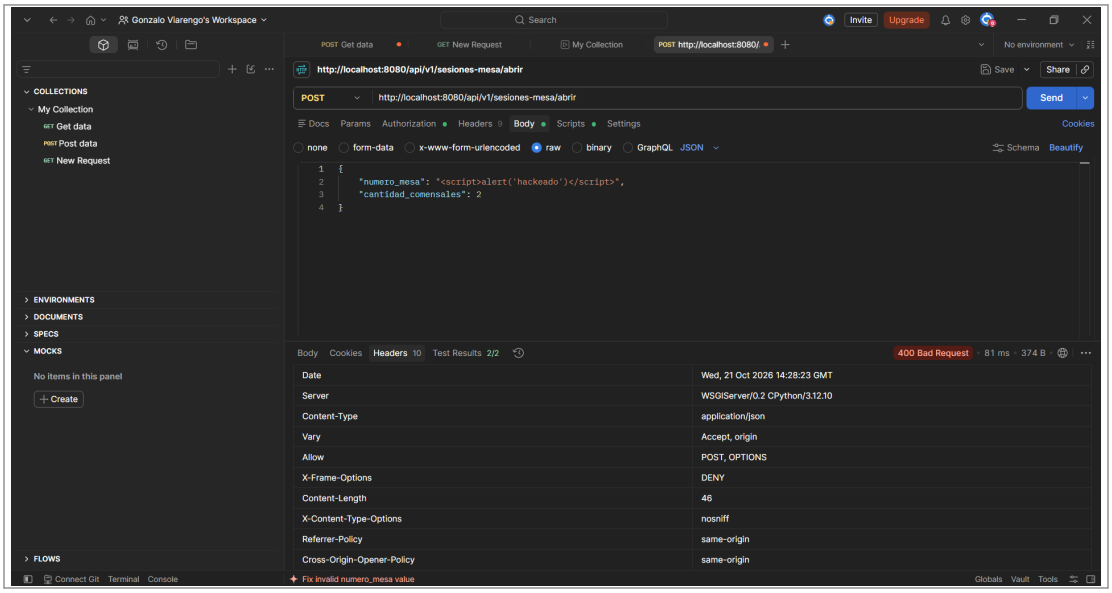


Captura 3. Test Results: Status 400 Bad Request y No es Error 500 en PASSED (2 de 2).

5. Prueba 3: headers de seguridad

Para proteger la aplicación contra vulnerabilidades del lado del cliente, se configuraron los middlewares de seguridad de Django. Para comprobarlo se volvió a enviar el request de la Prueba 2 y se abrió la pestaña Headers de la respuesta, que devolvió 10 encabezados.

Esperado: encabezados anti-sniffing y anti-clickjacking presentes en la respuesta. Obtenido: `X-Content-Type-Options: nosniff`, que impide que el navegador interprete un archivo con un tipo distinto al declarado (sniffing), y `X-Frame-Options: DENY`, que impide que la página se cargue dentro de un frame o iframe de otro sitio (clickjacking). También figuran `Referrer-Policy: same-origin` y `Cross-Origin-Opener-Policy: same-origin`. Los encabezados aparecen incluso en una respuesta de error 400, por lo que no dependen de que el request sea exitoso.



Captura 4. Pestaña Headers de la respuesta: X-Frame-Options: DENY y X-Content-Type-Options: nosniff.

6. Conclusión

Las tres pruebas se cumplieron en Postman contra el servidor del proyecto. La API respondió a las 20 iteraciones del Runner sin errores, con una latencia media de 124 ms, y las iteraciones posteriores a la primera respondieron entre 12 y 16 ms gracias a la caché. La entrada con código Javascript se rechazó con 400 sin ningún error 500, y las respuestas incluyen los encabezados nosniff y DENY, que elevan el nivel de protección de los endpoints.